

Week 1: 게임 에이전트 기초 - Random → Greedy → Minimax

게임 알고리즘 & 강화학습 고등학생 과정

학습 목표

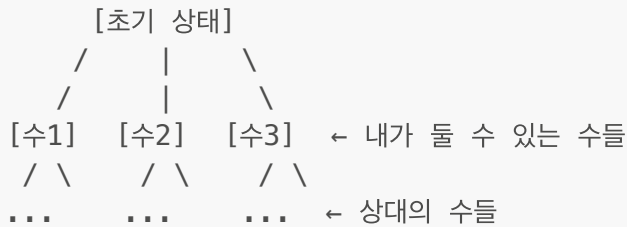
이번 주차에서는 게임 AI의 기초가 되는 세 가지 에이전트를 배웁니다.

- 게임을 트리 구조로 이해하기
- 평가 함수의 필요성과 설계
- Random, Greedy, Minimax 알고리즘의 원리와 차이점
- 실전 예제: 세균 전쟁(ATAXX) 게임

1. 게임 트리와 상태 공간

1.1 게임을 트리로 표현하기

게임은 하나의 **트리(tree)** 구조로 표현할 수 있습니다.



핵심 개념:

- **상태(State):** 게임판의 현재 상황 (돌의 배치, 누구 차례인지 등)
- **행동(Action):** 플레이어가 취할 수 있는 수 (예: A3 위치에 돌 놓기)
- **전이(Transition):** 어떤 행동을 했을 때 상태가 어떻게 변하는지

1.2 세균 전쟁(ATAXX) 게임 소개

이번 주 실습에서 사용할 게임은 **ATAXX**입니다.

게임판 구성:

	0	1	2	3	4	5	6
0	.	.	.	.	.	.	.
1	.	.	.	.	.	.	.
2	.	0	.	.	.	X	.
3	.	.	.	.	.	.	.
4	.	X	.	.	.	0	.

```

5 . . . . .
6 . . . . .

```

게임 규칙:

1. 보드: 7×7 격자
2. 플레이어: 두 명 (O, X)
3. 시작 위치:
 - O: (2,1), (4,5)
 - X: (2,5), (4,1)

4. 이동 방식:

- **분열 (거리 1):** 인접한 8방향으로 복제

```

. . .      . 0 .
. 0 .  →  0 0 .
. . .      . . .

```

- **도약 (거리 2):** 거리 2까지 이동 (원래 위치는 사라짐)

```

. . . . .      . . 0 . .
. . . . .      . . . . .
. 0 . . .  →  . . . . .
. . . . .      . . . . .

```

5. 감염: 이동/복제 후 인접 8방향의 적군이 내 색으로 변환

```

X X .      0 0 .
X 0 .  →  0 0 .
. . .      . . .
(0가 이동한 경우)

```

6. 게임 종료 조건:

- 한쪽 세균이 모두 소멸
- 빈칸이 없어짐
- 400턴 초과 (무승부)

7. 승리 조건: 게임 종료 시 더 많은 돌을 가진 플레이어

2. 평가 함수 (Evaluation Function)

2.1 왜 평가 함수가 필요한가?

게임 트리를 **끝까지** 탐색할 수 있다면 완벽한 플레이가 가능합니다. 하지만:

- 체스: 약 10^{120} 개의 가능한 게임
- 바둑: 약 10^{170} 개의 가능한 게임
- ATAXX: 수십억 개의 가능한 게임

→ **현실적으로 불가능!**

따라서 중간 상태를 **평가**하여 "이 상태가 얼마나 유리한지" 점수를 매겨야 합니다.

2.2 가장 간단한 평가 함수

ATAXX에서 가장 직관적인 평가:

```
def evaluate(board, player):  
    my_count = count_pieces(board, player)  
    opp_count = count_pieces(board, opponent(player))  
    return my_count - opp_count
```

예시:

```
현재 보드: O 15개, X 10개  
→ O 입장에서 평가: +5  
→ X 입장에서 평가: -5
```

이것만으로도 꽤 강력한 에이전트를 만들 수 있습니다!

3. Random Agent (무작위 에이전트)

3.1 알고리즘

```
def random_agent(board, player):  
    moves = get_legal_moves(board, player) # 가능한 수 목록  
    return random.choice(moves)           # 무작위로 하나 선택
```

3.2 특징

장점:

- 구현이 매우 간단
- 예측 불가능 (때로는 장점)

단점:

- 명백히 나쁜 수도 선택
- 승률이 매우 낮음

역할:

- **Baseline:** 다른 에이전트의 성능을 측정하는 기준점
- "Random보다 못하면 알고리즘이 잘못된 것"

4. Greedy Agent (탐욕 에이전트)

4.1 알고리즘

"1수 앞만 보고" 가장 좋은 수를 선택합니다.

```
def greedy_agent(board, player):
    moves = get_legal_moves(board, player)
    best_move = None
    best_score = -infinity

    for move in moves:
        # 이 수를 두었을 때의 보드 상태
        new_board = make_move(board, move, player)

        # 평가 함수로 점수 계산
        score = evaluate(new_board, player)

        if score > best_score:
            best_score = score
            best_move = move

    return best_move
```

4.2 예시

현재 상태: O 10개, X 10개

가능한 수 3가지:

- 수 A → O 11개, X 10개 (점수: +1)
- 수 B → O 13개, X 9개 (점수: +4) ← 선택!
- 수 C → O 10개, X 10개 (점수: 0)

Greedy는 수 B를 선택

4.3 성능

실험 결과: Greedy vs Random

- 23승 0패

- Random보다 압도적으로 강함!

한계:

- "1수 앞"만 봄
- 상대의 대응을 고려하지 않음
- 함정에 빠질 수 있음

5. Minimax 알고리즘

5.1 핵심 아이디어

"상대도 최선을 다한다"고 가정하자!

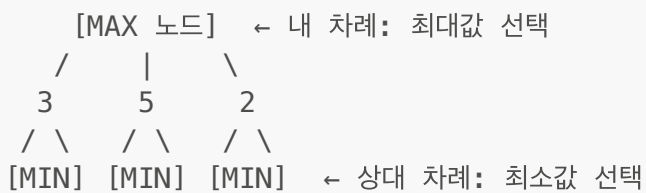
Greedy의 문제점:

내가 수 A를 두면 +5점! ← 좋아 보임
→ 하지만 상대가 대응하면 -10점으로 역전당함

Minimax의 해결책:

내가 수 A를 두면 +5점
→ 상대의 최선 대응을 고려하면 실제로는 -3점
내가 수 B를 두면 +3점
→ 상대의 최선 대응을 고려해도 +2점 유지 ← 이게 진짜 좋은 수!

5.2 MAX 노드와 MIN 노드



- **MAX 노드** (내 차례): 자식 중 **최댓값** 선택
- **MIN 노드** (상대 차례): 자식 중 **최솟값** 선택
 - 상대는 나에게 최악인 수를 선택할 것

5.3 의사코드

```
def minimax(board, depth, is_max_player):
    # 종료 조건
    if depth == 0 or game_over(board):
        return evaluate(board, original_player)
```

```

if is_max_player: # MAX 노드 (내 차례)
    max_eval = -infinity
    for move in get_legal_moves(board, current_player):
        new_board = make_move(board, move)
        eval = minimax(new_board, depth-1, False)
        max_eval = max(max_eval, eval)
    return max_eval

else: # MIN 노드 (상대 차례)
    min_eval = +infinity
    for move in get_legal_moves(board, opponent):
        new_board = make_move(board, move)
        eval = minimax(new_board, depth-1, True)
        min_eval = min(min_eval, eval)
    return min_eval

```

5.4 깊이 제한 (Depth Limit)

실전에서는 **깊이 제한**을 설정합니다:

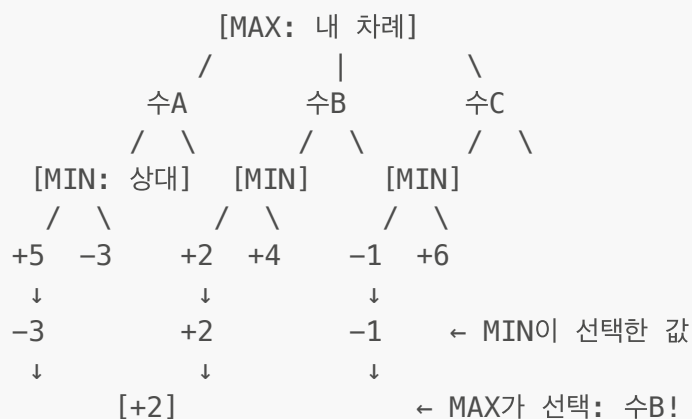
- **depth=1**: Greedy와 동일
- **depth=2**: 내 수 → 상대 수까지 고려
- **depth=4**: 내 수 → 상대 수 → 내 수 → 상대 수

탐색 노드 수:

가능한 수가 평균 30개라면:
 depth=2: 약 900개 노드
 depth=4: 약 810,000개 노드
 depth=6: 약 729,000,000개 노드 (!)

→ 깊이를 높일수록 강해지지만 느려짐

5.5 탐색 과정 예시



6. Negamax (Minimax 간소화 버전)

6.1 수학적 원리

핵심 아이디어:

$$\min(a, b) = -\max(-a, -b)$$

증명:

- 내 입장에서 +5점 = 상대 입장에서 -5점
- 상대가 최소값을 선택 = 부호를 바꾸면 최대값 선택

6.2 장점

Minimax에서는 MAX/MIN을 구분해야 하지만, Negamax는:

- 항상 최댓값만 선택
- 평가값에 부호만 바꿔주면 됨
- 코드가 훨씬 간결

6.3 의사코드

```
def negamax(board, depth, player):
    # 종료 조건
    if depth == 0 or game_over(board):
        return evaluate(board, player)

    max_eval = -infinity
    for move in get_legal_moves(board, player):
        new_board = make_move(board, move, player)

        # 재귀 호출: 상대 입장에서 평가 후 부호 반전!
        eval = -negamax(new_board, depth-1, opponent(player))

        max_eval = max(max_eval, eval)

    return max_eval
```

6.4 Minimax vs Negamax 비교

항목	Minimax	Negamax
조건 분기	MAX/MIN 구분 필요	항상 MAX
코드 길이	상대적으로 김	짧고 간결
성능	동일	동일

항목	Minimax	Negamax
이해 난이도	직관적	약간 추상적

실전에서는 **Negamax**가 더 많이 사용됩니다.

7. 성능 비교

7.1 실험 결과

Random vs Greedy:

- 결과: Greedy 23승 0패
- 결론: Greedy가 압도적으로 강함

Greedy vs Minimax(depth=2):

- 결과: Minimax 다수 승리
- 결론: 상대의 수를 고려하는 것이 중요

Minimax depth 비교:

Depth	특징	강도	속도
1	Greedy와 동일	약함	매우 빠름
2	상대 1수 고려	중간	빠름
4	2수씩 교대 고려	강함	느림
6	3수씩 교대 고려	매우 강함	매우 느림

7.2 SPRT (Sequential Probability Ratio Test)

게임 AI 성능을 통계적으로 검증하는 방법:

개념:

- "에이전트 A가 B보다 강한가?"를 적은 게임 수로 판정
- 100판을 다 두지 않아도 50판 정도면 결론 가능

결과 해석:

- H0 (귀무가설): 두 에이전트가 비슷함
- H1 (대립가설): A가 B보다 강함
- SPRT는 빠르게 H0 또는 H1을 채택

예시:

Greedy vs Random:
 - 10판: Greedy 10승 → H1 채택 (Greedy가 강함)

9 / 10

문제 1: 개념 확인

다음 상황에서 Greedy와 Minimax의 선택이 다를 수 있는 이유를 설명하시오.

현재: O 10개, X 10개
 수 A: 즉시 O 15개, X 9개 (Greedy 선호)
 수 B: O 12개, X 10개 (Minimax 선호)

문제 2: 게임 트리

다음 게임 트리에서 Minimax가 선택할 수는?



(각 리프 노드는 MIN이 선택한 결과)

문제 3: 평가 함수

ATAXX에서 다음 두 평가 함수 중 어느 것이 더 좋을까?

```

# 평가 1
score = my_count - opp_count

# 평가 2
score = my_count - opp_count + mobility_bonus
# mobility_bonus: 내가 둘 수 있는 수의 개수
  
```

참고 자료

- 게임 트리 탐색: <https://infossm.github.io/blog/2025/10/25/adv-game-search/>
- Minimax 알고리즘: Wikipedia
- ATAXX 게임 규칙: 수업 코드 참조

다음 시간: Alpha-Beta Pruning과 탐색 최적화

Week 1 완료! 수고하셨습니다.